

Prueba Técnica:

Hay 2 opciones para la prueba técnica, realizar solo 1 de ellas. Todo el código debe ser subido a un repositorio público en github después de ser analizado se solicitará una reunión corta donde se debe compartir la prueba técnica funcionando.

No hay como tal tiempo para terminar la prueba técnica, pero si no se envía el link del repositorio público antes de finalizar el mes de Junio, se asumirá por finalizada la prueba.

Importante: NO se permite el uso de Inteligencia Artificial para la generación de código de esta prueba.



OPCIÓN A: Backend (Python - Django o Flask)

Objetivo: Crear una API REST para un sistema de agendamiento interno con reglas de negocio muy específicas.

El Reto: "El Algoritmo de Bloqueo Anti-Solapamiento"

Debes crear un endpoint para registrar citas de un CRM.

Requisitos Funcionales:

1. **Crear Cita (POST /api/appointments/):** Debe recibir `client_id`, `date` (fecha), `start_time` (hora inicio) y `duration_minutes` (duración).
2. **Reglas de Validación Críticas:**
 - **Regla del Almuerzo:** No se pueden agendar citas que toquen el rango de 13:00 a 14:00 (ej. una cita de 12:30 a 13:15 debe ser rechazada).
 - **Regla del Buffer de Cliente VIP:** Si el cliente tiene una categoría "VIP" (simulado o hardcodeado en la base de datos), el sistema debe bloquear **automáticamente 15 minutos antes y 15 minutos después** de su cita. Ninguna otra cita puede solaparse con este "tiempo de colchón".
 - **Bloqueo de Martes:** Los martes por la tarde (después de las 16:00) el sistema está en mantenimiento interno; no se permiten citas.
3. **Listar Disponibilidad (GET /api/appointments/availability/?date=YYYY-MM-DD):** Debe retornar los bloques de tiempo que ya están ocupados o bloqueados por las reglas anteriores para ese día.

Restricciones Técnicas:

- No usar librerías externas de manejo de calendarios (como `scheduler` o similares). Todo el cálculo de solapamiento de horas debe hacerse con lógica nativa de Python (`datetime`).

- Uso de Base de Datos en memoria (SQLite o diccionarios globales si es Flask).



OPCIÓN B: Frontend (JavaScript - React.js)

Objetivo: Construir la interfaz de un módulo de envío de correo masivo conectado a un CRM.

El Reto: "La Cola de Envíos Dinámica"

Crear una interfaz interactiva para seleccionar clientes y preparar una campaña de correo masivo, controlando los estados de manera estricta.

Requisitos Funcionales:

1. **Lista de Clientes:** Renderizar una lista de 10 clientes (puedes usar un array hardcodeado) con campos: Nombre, Correo, Estado (Activo/Inactivo) y Puntuación de Riesgo (1 al 100).
2. **Selección Condicional :**
 - El usuario puede seleccionar múltiples clientes usando *checkboxes* para añadirlos a la "Cola de Envío".
 - **Restricción:** Si el usuario selecciona un cliente con una Puntuación de Riesgo superior a 80, automáticamente se deben deshabilitar temporalmente todos los clientes "Inactivos" de la lista para evitar envíos inseguros.
3. **Simulador de Envío Secuencial:**
 - Al hacer clic en "Iniciar Campaña", la aplicación debe simular el envío de correos **uno por uno** (con un `setTimeout` de 1.5 segundos por cliente).
 - Se debe mostrar visualmente el progreso: "Enviando a [Correo]...", "Éxito 
 o "Fallo .
- **Simulación de Error:** Los clientes cuyo correo termine en `.org` deben fallar a propósito para probar el manejo de errores en la interfaz.

Restricciones Técnicas:

- **Prohibido** usar librerías de componentes (No Material UI, No Tailwind, No Ant Design). Debes usar CSS puro o CSS Modules.
- Prohibido usar librerías de manejo de estado global (No Redux, No Zustand). Todo con `useState` y `useEffect`.